

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

**«САРАТОВСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ИМЕНИ Н.Г.ЧЕРНЫШЕВСКОГО»**

Кафедра математической теории упругости и биомеханики

Разработка графического интерфейса пользователя для библиотеки

КУРСОВАЯ РАБОТА

студента 2 курса 242 группы
направления 09.03.03 – Прикладная информатика

механико-математического факультета

Афанасьева Даниила Алексеевича

Научный руководитель
профессор, д.ф.-м.н., доцент

И.В. Папкина

Зав. кафедрой
зав. кафедрой, д.ф.-м.н., профессор

Л.Ю. Коссович

Саратов 2026

СОДЕРЖАНИЕ

Стр.

ВВЕДЕНИЕ	3
1 Проектирование базы данных	4
1.1 Анализ предметной области	4
1.2 Структура таблиц	6
2 Моделирование функциональных требований	8
2.1 Сценарии использования	9
3 Реализация модуля базы данных	11
4 Реализация графического интерфейса	14
5 Тестирование приложения	17
5.1 Функциональное тестирование	17
5.2 Тестирование защиты данных	18
ЗАКЛЮЧЕНИЕ	19
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	20

ВВЕДЕНИЕ

В современном мире автоматизация процессов управления становится неотъемлемой частью эффективной работы любой организации. Библиотеки, как учреждения, работающие с большим объёмом информации о книгах и читателях, особенно нуждаются в специализированных программных решениях. Применение информационных систем позволяет существенно сократить время на выполнение рутинных операций, снизить вероятность ошибок и обеспечить надёжное хранение данных.

Актуальность данной работы обусловлена необходимостью автоматизации финансового учёта в библиотеке, предоставляющей книги напрокат. Ручное ведение записей о выданных книгах, залоговых суммах и платежах за прокат сопряжено с высоким риском ошибок и значительными временными затратами. Разработка графического интерфейса пользователя (GUI) на основе современных технологий позволяет решить эти проблемы [1].

Целью данной работы является разработка десктопного приложения с графическим интерфейсом для автоматизации учёта проката книг в библиотеке, включая управление фондом книг, картотекой читателей и финансовыми показателями.

Для достижения цели решаются следующие задачи:

- анализ предметной области и проектирование структуры базы данных;
- разработка диаграмм UML и ER для формализации требований;
- реализация модуля работы с базой данных на языке Python;
- разработка графического интерфейса с использованием библиотеки PyQt5;
- тестирование функциональности приложения.

В разделе 1 описывается проектирование базы данных и приводится ER-диаграмма. В разделе 2 рассматриваются функциональные требования к системе с использованием UML-диаграммы вариантов использования. В разделе 3 описывается реализация приложения. В разделе 4 рассматривается структура интерфейса. Раздел 5 посвящён тестированию.

1 Проектирование базы данных

1.1 Анализ предметной области

Система предназначена для библиотеки, осуществляющей прокат книг на коммерческой основе. Такая модель работы предполагает взимание платы за пользование книгами и внесение залоговой суммы, возвращаемой при возврате книги в сохранности.

Предметная область включает следующие сущности и процессы:

Книги — единицы фонда, которые могут быть выданы напрокат. Каждая книга характеризуется названием, автором, жанром, залоговой стоимостью (которая взимается при выдаче и возвращается при возврате), стоимостью проката (плата за пользование книгой) и количеством экземпляров. Библиотека может иметь несколько экземпляров одной книги, что требует контроля их доступности. При проектировании системы важно обеспечить механизм отслеживания количества свободных экземпляров в реальном времени.

Читатели — зарегистрированные клиенты библиотеки с анкетными данными (фамилия, имя, отчество, адрес, телефон). Ведение картотеки читателей позволяет идентифицировать клиентов, отслеживать историю их взаимодействия с библиотекой, формировать статистику наиболее активных пользователей. Также наличие контактных данных обеспечивает возможность связи с читателем в случае просрочки возврата книги.

Жанры — классификатор литературных жанров, используемый для категоризации книг. Система жанров облегчает поиск и систематизацию фонда, позволяет формировать отчёты о популярности различных категорий литературы. Классификация может включать такие жанры, как художественная литература, научная литература, детективы, фантастика и другие.

Выдачи — факты выдачи книги читателю с указанием дат выдачи и ожидаемого возврата. Каждая выдача связывает конкретную книгу с конкретным читателем и фиксирует временные рамки проката. Система должна обеспечивать контроль за своевременностью возврата и уведомлять о просроченных выдачах.

Возвраты — факты возврата книги с фиксацией суммы оплаченного проката. При возврате система рассчитывает стоимость проката на основе уста-

новленной для книги цены, фиксирует дату возврата и формирует запись о финансовой транзакции. Эти данные используются для формирования финансовой отчётности и анализа доходности библиотечного фонда.

Основные бизнес-процессы, реализуемые системой:

1. Регистрация читателя — внесение в систему данных нового клиента библиотеки.
2. Пополнение фонда — добавление новых книг с указанием всех необходимых характеристик.
3. Выдача книги — оформление проката с проверкой доступности экземпляров.
4. Возврат книги — приём книги, расчёт и фиксация оплаты проката.
5. Формирование отчётности — генерация отчётов о текущих выдачах и финансовых показателях.

При анализе предметной области были выявлены следующие ограничения и правила:

- Книгу можно выдать только при наличии свободных экземпляров.
- Один читатель может иметь несколько активных выдач одновременно.
- Возврат книги возможен только после её выдачи, повторный возврат блокируется системой.
- Стоимость проката фиксируется на момент выдачи книги и не изменяется при возврате.

На рисунке 1.1 представлена ER-диаграмма, отображающая структуру базы данных с таблицами и связями между ними.

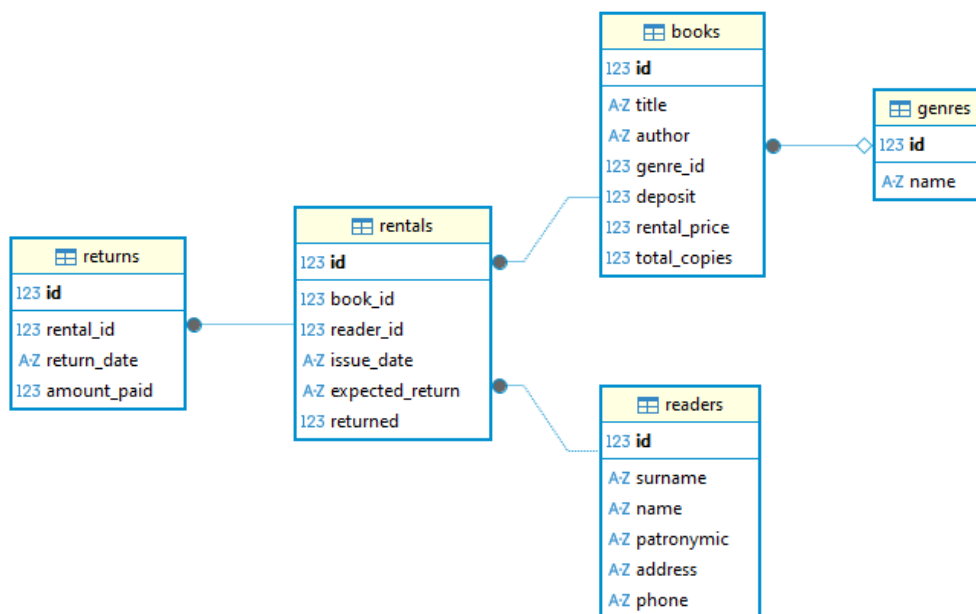


Рисунок 1.1 — ER-диаграмма базы данных системы учёта проката книг

1.2 Структура таблиц

База данных реализована в СУБД SQLite [2] и содержит пять таблиц.

Таблица **genres** хранит жанры книг и содержит поля: **id** (первичный ключ, автоинкремент) и **name** (уникальное название жанра).

Таблица **books** хранит сведения о книгах: **id**, **title** (название), **author** (автор), **genre_id** (внешний ключ на таблицу жанров), **deposit** (залоговая стоимость), **rental_price** (стоимость проката), **total_copies** (общее количество экземпляров).

Таблица **readers** содержит картотеку читателей: **id**, **surname**, **name**, **patronymic**, **address**, **phone**.

Таблица **rentals** фиксирует факты выдачи книг: **id**, **book_id** (внешний ключ), **reader_id** (внешний ключ), **issue_date** (дата выдачи), **expected_return** (ожидаемая дата возврата), **returned** (флаг возврата).

Таблица **returns** хранит финансовую историю возвратов: **id**, **rental_id** (внешний ключ), **return_date** (дата возврата), **amount_paid** (сумма оплаты).

В таблице 1.1 представлены связи между таблицами базы данных.

Таблица 1.1 — Связи между таблицами базы данных

Таблица	Связанная таблица	Тип связи
genres	books	Один ко многим (один жанр — много книг)
books	rentals	Один ко многим (одна книга — много выдач)
readers	rentals	Один ко многим (один читатель — много выдач)
rentals	returns	Один к одному (одна выдача — один возврат)

Ссылочная целостность обеспечивается внешними ключами с политиками ON UPDATE CASCADE и ON DELETE CASCADE или SET NULL.

2 Моделирование функциональных требований

Основные варианты использования системы сгруппированы по функциональным областям.

Управление жанрами: добавление нового жанра; редактирование названия жанра; удаление жанра; просмотр списка жанров.

Управление книгами: добавление книги с указанием названия, автора, жанра, залоговой стоимости, стоимости проката и количества экземпляров; редактирование данных книги; удаление книги; просмотр каталога с информацией о доступных экземплярах.

Управление читателями: регистрация нового читателя; редактирование анкетных данных; удаление читателя из картотеки; просмотр списка читателей.

Учёт выдачи книг: оформление выдачи книги читателю с указанием дат; оформление возврата книги с автоматическим расчётом суммы оплаты; просмотр текущих и завершённых выдач.

Финансовый учёт и отчётность: просмотр истории возвратов и оплат; отчёт по текущим выдачам (книги на руках); отчёт по выручке в разрезе книг.

Для формализации функциональных требований была построена диаграмма вариантов использования (UML Use Case Diagram) [3], представленная на рисунке 2.1. Диаграмма визуализирует основные функции системы и их взаимосвязь с актором.

В системе определён один актор — сотрудник библиотеки, который взаимодействует со всеми функциями приложения. Потенциально систему можно расширить, введя роли (администратор, бухгалтер) с разными уровнями доступа, но в текущей реализации все функции доступны единственному пользователю.



Рисунок 2.1 — Диаграмма вариантов использования системы учёта проката книг

2.1 Сценарии использования

Рассмотрим ключевые сценарии взаимодействия пользователя с системой.

Сценарий 1: Выдача книги читателю.

1. Пользователь переходит на вкладку «Выдачи».
2. Нажимает кнопку «Выдать книгу».
3. В диалоговом окне выбирает книгу из списка доступных экземпляров.
4. Выбирает читателя из картотеки.
5. Указывает дату выдачи и ожидаемую дату возврата.
6. Система сохраняет запись о выдаче в таблице `rentals`.

7. Количество доступных экземпляров книги уменьшается на 1.

Сценарий 2: Оформление возврата книги.

1. Пользователь выбирает активную запись о выдаче в таблице.
2. Нажимает кнопку «Оформить возврат».
3. Система отображает информацию о книге, читателе и стоимости проката.
4. После подтверждения фиксируется дата возврата, записывается сумма оплаты, статус выдачи обновляется на «Возвращена».
5. Таблица «Возвраты» обновляется автоматически.

Сценарий 3: Просмотр финансового отчёта.

1. Пользователь переходит на вкладку «Отчёты».
2. Система отображает список книг, находящихся на руках у читателей.
3. Система отображает сводную таблицу выручки по каждой книге.
4. При необходимости пользователь нажимает «Обновить отчёты» для получения актуальных данных.

3 Реализация модуля базы данных

Приложение построено по принципу разделения ответственности [4] и состоит из двух основных модулей:

- `database.py` — модуль для взаимодействия с базой данных SQLite; содержит класс `Database` с методами CRUD-операций для всех таблиц;
- `main_qt.py` — основной модуль приложения; содержит классы вкладок интерфейса и главного окна, реализованные средствами PyQt5.

Такая архитектура обеспечивает:

- Модульность: каждый компонент выполняет чётко определённую функцию.
- Тестируемость: слой данных можно тестировать независимо от интерфейса.
- Расширяемость: легко добавлять новые функции или изменять существующие без затрагивания других модулей.
- Повторное использование: класс `Database` можно использовать в других интерфейсах (например, веб-интерфейсе или консольном приложении).

Класс `Database` инкапсулирует все операции с базой данных [2]. При инициализации объекта вызывается метод `_init_db()`, который создаёт все таблицы при их отсутствии и наполняет базу тестовыми данными, если таблицы пусты.

Метод `available_copies(book_id)` вычисляет количество доступных экземпляров книги путём вычитания числа невозвращённых выдач из общего количества экземпляров. Это позволяет предотвратить выдачу книги при отсутствии свободных экземпляров.

Метод `process_return(rental_id, return_date)` реализует бизнес-логику возврата: определяет стоимость проката конкретной книги, создаёт запись в таблице `returns` и обновляет флаг `returned` в таблице `rentals`. Метод возвращает сумму оплаты для отображения пользователю.

Схема инициализации базы данных представлена ниже.

```

1      CREATE TABLE IF NOT EXISTS genres (
2      id    INTEGER PRIMARY KEY AUTOINCREMENT,
3      name TEXT NOT NULL UNIQUE
4      );
5      CREATE TABLE IF NOT EXISTS books (
6      id          INTEGER PRIMARY KEY AUTOINCREMENT,
7      title       TEXT NOT NULL,
8      author      TEXT NOT NULL,
9      genre_id    INTEGER REFERENCES genres(id) ON UPDATE CASCADE ON
10     ⇨ DELETE SET NULL,
11      deposit     REAL NOT NULL DEFAULT 0,
12      rental_price REAL NOT NULL DEFAULT 0,
13      total_copies INTEGER NOT NULL DEFAULT 1
14     );
15     CREATE TABLE IF NOT EXISTS readers (
16     id          INTEGER PRIMARY KEY AUTOINCREMENT,
17     surname     TEXT NOT NULL,
18     name        TEXT NOT NULL,
19     patronymic  TEXT NOT NULL DEFAULT '',
20     address     TEXT NOT NULL DEFAULT '',
21     phone       TEXT NOT NULL DEFAULT ''
22     );
23     CREATE TABLE IF NOT EXISTS rentals (
24     id          INTEGER PRIMARY KEY AUTOINCREMENT,
25     book_id     INTEGER NOT NULL REFERENCES books(id) ON DELETE
26     ⇨ CASCADE,
27     reader_id   INTEGER NOT NULL REFERENCES readers(id) ON DELETE
28     ⇨ CASCADE,
29     issue_date  TEXT NOT NULL,
30     expected_return TEXT NOT NULL,
31     returned    INTEGER NOT NULL DEFAULT 0
32     );
33     CREATE TABLE IF NOT EXISTS returns (
34     id          INTEGER PRIMARY KEY AUTOINCREMENT,
35     rental_id   INTEGER NOT NULL REFERENCES rentals(id) ON DELETE
36     ⇨ CASCADE,
37     return_date TEXT NOT NULL,
38     amount_paid REAL NOT NULL DEFAULT 0
39     );

```

Далее представлена реализация метода оформления возврата книги, который реализует ключевую финансовую логику приложения.

```

1         def process_return(self, rental_id: int, return_date: str) ->
           ↪ float:
2         with self._connect() as conn:
3             rental = conn.execute(
4                 "SELECT * FROM rentals WHERE id=?", (rental_id,)
5             ).fetchone()
6             if not rental or rental['returned']:
7                 return 0.0
8             book = conn.execute(
9                 "SELECT rental_price FROM books WHERE id=?",
10                (rental['book_id'],)
11            ).fetchone()
12            amount = book['rental_price'] if book else 0.0
13            conn.execute(
14                "INSERT INTO returns(rental_id, return_date, amount_paid)"
15                " VALUES (?, ?, ?)",
16                (rental_id, return_date, amount)
17            )
18            conn.execute(
19                "UPDATE rentals SET returned=1 WHERE id=?", (rental_id,)
20            )
21            return amount

```

4 Реализация графического интерфейса

Графический интерфейс реализован с использованием библиотеки PyQt5 [5], которая предоставляет богатый набор виджетов и инструментов для создания кроссплатформенных приложений с нативным внешним видом. PyQt5 является набором привязок Python [6] к фреймворку Qt [7]. Данная библиотека обеспечивает полноценную интеграцию с операционными системами Windows, Linux и macOS, предоставляя единообразный программный интерфейс.

Приложение содержит шесть вкладок:

- Жанры — управление справочником литературных жанров. Данная вкладка позволяет добавлять новые категории книг, редактировать существующие названия и удалять неиспользуемые жанры.
- Книги — управление каталогом книг с информацией о доступности. Здесь отображается полная информация о каждой книге: название, автор, жанр, залоговая стоимость, цена проката и общее количество экземпляров. Система автоматически вычисляет количество доступных для выдачи экземпляров.
- Читатели — управление картотекой читателей. Вкладка содержит персональные данные всех зарегистрированных пользователей библиотеки, включая контактную информацию для связи.
- Выдачи — учёт выдач книг и оформление возвратов. Это ключевая вкладка с точки зрения бизнес-процессов, здесь происходит основная работа с транзакциями проката.
- Возвраты — история возвратов и финансовых операций. Вкладка предоставляет полную историю всех завершённых операций проката с указанием дат и сумм оплаты.
- Отчёты — сводные отчёты по активным выдачам и выручке. Аналитический раздел, позволяющий контролировать финансовые показатели работы библиотеки.

Каждая вкладка использует вспомогательную функцию `make_table()`, которая создаёт таблицу `QTableWidget` [5] с настроенными режимами отображения: запрет редактирования напрямую, выделение строк целиком, чередование цветов строк, автоматическое растяжение столбцов по ширине окна.

Для добавления и редактирования записей используются модальные диалоговые окна (QDialog). В приложении реализованы следующие диалоги:

- GenreDialog — ввод названия жанра;
- BookDialog — ввод данных книги с выбором жанра из выпадающего списка;
- ReaderDialog — ввод анкетных данных читателя;
- RentalDialog — оформление выдачи с выбором книги и читателя, указанием дат.

На рисунке 4.1 представлен внешний вид главного окна приложения.

ID	Книга	Читатель	Дата выдачи	Ожид. возврат	Статус	Залог (₽)
18	История России	Егорова Наталья ...	2026-05-01	2026-05-15	На руках	700.00
15	Война и мир	Новикова Елена ...	2026-04-29	2026-05-13	На руках	800.00
19	Мартин Иден	Афанасьев Даниил ...	2026-04-27	2026-05-11	На руках	420.00
14	Автостопом по галактике	Попов Андрей ...	2026-04-26	2026-05-10	На руках	350.00
13	1984	Петрова Мария Сергеевна	2026-04-24	2026-05-08	На руках	400.00
16	Граф Монте-Кристо	Михайлов Виктор ...	2026-04-19	2026-05-03	⚠ Просрочена	500.00
10	Дюна	Лебедев Николай ...	2026-04-16	2026-04-30	Возвращена	500.00
9	Думай медленно, решай...	Новикова Елена ...	2026-04-14	2026-04-28	Возвращена	400.00
17	Думай медленно, решай...	Козлов Дмитрий ...	2026-04-14	2026-04-28	⚠ Просрочена	400.00
8	Десять негритят	Иванов Иван Иванович	2026-04-09	2026-04-23	Возвращена	360.00
7	Граф Монте-Кристо	Афанасьев Даниил ...	2026-04-04	2026-04-18	Возвращена	500.00
6	Гиперион	Морозов Сергей ...	2026-03-30	2026-04-13	Возвращена	480.00
5	Война и мир	Егорова Наталья ...	2026-03-25	2026-04-08	Возвращена	800.00
4	Анна Каренина	Михайлов Виктор ...	2026-03-20	2026-04-03	Возвращена	750.00
3	Автостопом по галактике	Волкова Анастасия ...	2026-03-15	2026-03-29	Возвращена	350.00
2	Sherlock Holmes	Козлов Дмитрий ...	2026-03-10	2026-03-24	Возвращена	380.00
1	1984	Захарова Ольга ...	2026-03-05	2026-03-19	Возвращена	400.00
12	Имя розы	Волкова Анастасия ...	2026-02-13	2026-02-27	Возвращена	550.00
11	Евгений Онегин	Захарова Ольга ...	2026-02-03	2026-02-17	Возвращена	300.00

Рисунок 4.1 — Главное окно приложения, вкладка «Выдачи»

Вкладка «Выдачи» является ключевой с точки зрения бизнес-логики. В таблице отображаются все записи о выдачах с автоматическим расчётом статуса: «Возвращена», «На руках» или «Просрочена» (если текущая дата превысила ожидаемую дату возврата).

В диалоге выдачи книги в выпадающем списке отображаются только книги, у которых есть свободные экземпляры. При оформлении возврата производится запрос подтверждения с отображением суммы проката, после чего данные сохраняются в базе, а вкладка «Возвраты» обновляется автоматически.

Вкладка «Отчёты» формирует два отчёта: текущие выдачи (книги на руках у читателей с указанием залогов) и сводную таблицу выручки в разрезе каждой книги по всем завершённым операциям проката.

При переключении между вкладками данные автоматически обновляются благодаря подключению сигнала `currentChanged` вкладочного виджета к методу обновления активной вкладки.

5 Тестирование приложения

5.1 Функциональное тестирование

В процессе разработки было проведено функциональное тестирование всех операций приложения. В таблице 5.1 приведены результаты тестирования ключевых функций.

Таблица 5.1 — Результаты функционального тестирования

Тестируемая функция	Ожидаемый результат	Результат
Добавление жанра	Запись сохраняется, список обновляется	Пройден
Добавление книги с жанром	Книга добавлена, связь с жанром установлена	Пройден
Регистрация читателя	Читатель добавлен в картотеку	Пройден
Выдача книги (есть экземпляр)	Запись создана, число доступных экз. уменьшилось	Пройден
Выдача (нет свободных экз.)	Книга не отображается в списке выдачи	Пройден
Оформление возврата	Запись создана, стоимость рассчитана верно	Пройден
Повторный возврат	Выдаётся предупреждение без изменения данных	Пройден
Отчёт по выручке	Отчёт содержит корректные суммы	Пройден
Просроченные выдачи	Статус «Просрочена» при нарушении срока	Пройден

5.2 Тестирование защиты данных

Были проверены сценарии, связанные с обеспечением целостности данных. Попытка выдать книгу без свободных экземпляров блокируется на уровне интерфейса. Попытка повторно оформить возврат уже возвращённой книги вызывает информационное сообщение без изменения данных в базе. Ссылочная целостность таблиц поддерживается настройками внешних ключей SQLite с каскадным удалением зависимых записей.

Все тесты успешно пройдены, приложение функционирует корректно и соответствует заявленным требованиям.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была разработана информационная система для автоматизации учёта проката книг в библиотеке. Достигнуты следующие результаты:

1. Проведён анализ предметной области, определены основные сущности: жанры, книги, читатели, выдачи и возвраты.
2. Спроектирована реляционная база данных на основе СУБД SQLite [2,8], состоящая из пяти взаимосвязанных таблиц с обеспечением ссылочной целостности посредством внешних ключей.
3. Разработана ER-диаграмма базы данных [4] и диаграмма вариантов использования (UML Use Case) [3], формализующая функциональные требования [9].
4. Реализован модуль `database.py`, инкапсулирующий все операции с базой данных, включая бизнес-логику расчёта стоимости проката при возврате.
5. Разработан графический интерфейс пользователя на Python [6, 10] с использованием библиотеки PyQt5 [5], организованный в виде шести тематических вкладок.
6. Проведено функциональное тестирование, подтвердившее корректную работу всех реализованных функций.

Созданное приложение позволяет автоматизировать учёт выдачи книг, отслеживать финансовые показатели проката (суммы залогов, стоимость проката, общую выручку) и контролировать своевременность возврата книг.

Перспективы развития системы включают: добавление экспорта отчётов в форматы PDF и Excel; реализацию поиска и фильтрации; добавление функции уведомлений о просроченных выдачах; разработку веб-версии приложения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 Прохоренок, Н. А. Python 3 и PyQt 5. Разработка приложений / Н. А. Прохоренок. — СПб. : БХВ-Петербург, 2016. — 832 с.
- 2 SQLite. SQLite Home Page [Электронный ресурс]. — URL:<https://www.sqlite.org/index.html> (дата обращения: 01.05.2026). - Загл. с экрана. - Яз. англ.
- 3 Fowler, M. UML Distilled: A Brief Guide to the Standard Object Modeling Language / M. Fowler. — 3rd ed. — Addison-Wesley Professional, 2004. — 192 p. - Яз. англ.
- 4 Коннолли, Т. Базы данных. Проектирование, реализация и сопровождение / Т. Коннолли, К. Бегг. — 3-е изд. — М. : Вильямс, 2003. — 1440 с.
- 5 Riverbank Computing Limited. PyQt5 Reference Guide [Электронный ресурс]. — URL:<https://www.riverbankcomputing.com/static/Docs/PyQt5/> (дата обращения: 01.05.2026). - Загл. с экрана. - Яз. англ.
- 6 Python Software Foundation. Python 3 Documentation [Электронный ресурс]. — URL:<https://docs.python.org/3/> (дата обращения: 01.05.2026). - Загл. с экрана. - Яз. англ.
- 7 Summerfield, M. Rapid GUI Programming with Python and Qt / M. Summerfield. — Addison-Wesley Professional, 2007. — 648 p. - Яз. англ.
- 8 Дейт, К. Дж. Введение в системы баз данных / К. Дж. Дейт. — 8-е изд. — М. : Вильямс, 2006. — 1328 с.
- 9 ГОСТ 34.602-89. Техническое задание на создание автоматизированной системы : Введ. 1990-01-01. — М. : Издательство стандартов, 1989. — 24 с.
- 10 Культин, Н. Б. Основы программирования в Python 3 / Н. Б. Культин. — СПб. : БХВ-Петербург, 2018. — 288 с.